https://github.com/dparada2020225/lab-12-teoria

Denil Parada 24761

Lab #12

Problema 1: 25%

- Escriba la reducción-β de la operación lógica NOT
- Escriba y explique como se vería la recursión y los ciclos.
- Explique cuando es prudente usar este tipo de programación y cuando no. De un ejemplo para cada caso.

a) Reducción-β de la operación NOT

Regla de la reducción-β:

$(\lambda x.\ M)\ N \rightarrow_\beta M[x := N]$

"cada x libre dentro de M se cambia por N"

Booleanos de Church (un booleano elige entre 2 cosas):

$TRUE = \lambda x.\lambda y.\ x$ (se queda con la 1ra)

$FALSE = \lambda x.\lambda y.\ y$ (se queda con la 2da)

NOT tiene que devolver FALSE si p es TRUE y TRUE si p es FALSE, entonces se usa a p para elegir:

$NOT = \lambda p.\ p\ FALSE\ TRUE$

Caso 1: NOT TRUE

$NOT\ TRUE = (\lambda p.\ p\ FALSE\ TRUE)\ (\lambda x.\lambda y.\ x)$

$\rightarrow_\beta (\lambda x.\lambda y.\ x)\ FALSE\ TRUE \quad [p := TRUE]$

$\rightarrow_\beta (\lambda y.\ FALSE)\ TRUE \quad [x := FALSE]$

$\rightarrow_\beta FALSE \quad [y := TRUE,\ y\ no\ aparece]$

$= \lambda x.\lambda y.\ y \quad \therefore NOT\ TRUE = FALSE\ \checkmark$

Caso 2: NOT FALSE

NOT FALSE = $(\lambda p.\ p\ \text{FALSE}\ \text{TRUE})\ (\lambda x.\lambda y.\ y)$

$\rightarrow_\beta (\lambda x.\lambda y.\ y)\ \text{FALSE}\ \text{TRUE}$ [p := FALSE]

$\rightarrow_\beta (\lambda y.\ y)\ \text{TRUE}$ [x := FALSE, x no aparece]

$\rightarrow_\beta \text{TRUE}$ [y := TRUE]

$= \lambda x.\lambda y.\ x$ ∴ NOT FALSE = TRUE ✓

Caso 3: NOT TRUE con todo expandido

(se usan a, b dentro de NOT para no confundir variables)

$(\lambda p.\ p\ (\lambda a.\lambda b.\ b)\ (\lambda a.\lambda b.\ a))\ (\lambda x.\lambda y.\ x)$

$\rightarrow_\beta (\lambda x.\lambda y.\ x)\ (\lambda a.\lambda b.\ b)\ (\lambda a.\lambda b.\ a)$ [p := TRUE]

$\rightarrow_\beta (\lambda y.\ \lambda a.\lambda b.\ b)\ (\lambda a.\lambda b.\ a)$ [x := $\lambda a.\lambda b.\ b$]

$\rightarrow_\beta \lambda a.\lambda b.\ b$ [y := $\lambda a.\lambda b.\ a$, no aparece]

$\equiv \lambda x.\lambda y.\ y = \text{FALSE}$ (renombrar a,b es conversión-α) ✓

Otra forma: NOT que intercambia las opciones

$\text{NOT} = \lambda p.\lambda a.\lambda b.\ p\ b\ a$

$\text{NOT TRUE} = (\lambda p.\lambda a.\lambda b.\ p\ b\ a)\ (\lambda x.\lambda y.\ x)$

$\rightarrow_\beta \lambda a.\lambda b.\ (\lambda x.\lambda y.\ x)\ b\ a$ [p := TRUE]

$\rightarrow_\beta \lambda a.\lambda b.\ (\lambda y.\ b)\ a$ [x := b]

$\rightarrow_\beta \lambda a.\lambda b.\ b \equiv \text{FALSE}$ ✓ [y := a]

Comprobación:

$p = \text{TRUE} \Rightarrow \text{NOT}\ p = \text{FALSE}$ ✓

$p = \text{FALSE} \Rightarrow \text{NOT}\ p = \text{TRUE}$ ✓

NOT (NOT TRUE) = NOT FALSE = TRUE ✓

b) Recursión y ciclos

En el cálculo λ (y en programación funcional pura):

- las funciones son anónimas: no tienen nombre para llamarse
- no hay variables que cambien ni for / while

Por eso la recursión se logra con un combinador de punto fijo y los ciclos se escriben como recursión.

Combinador Y:

$$Y = \lambda f.\ (\lambda x.\ f\ (x\ x))\ (\lambda x.\ f\ (x\ x))$$

Propiedad: $Y\ g = g\ (Y\ g)$ ($Y\ g$ es punto fijo de g)

Paso 1: $Y\ g \to_\beta (\lambda x.\ g\ (x\ x))\ (\lambda x.\ g\ (x\ x))$ $[f := g]$

Paso 2: $\to_\beta g\ ((\lambda x.\ g\ (x\ x))\ (\lambda x.\ g\ (x\ x)))$

$[x := \lambda x.\ g\ (x\ x)]$

Paso 3: lo de adentro de g es el resultado del Paso 1, o sea $Y\ g$

$\therefore\ Y\ g = g\ (Y\ g)$

Así g se recibe "a sí misma" como parámetro y se puede volver a llamar: $g\ (g\ (g\ (\ldots)))$

Recursión: factorial

$$G = \lambda f.\lambda n.\ IF\ (ISZERO\ n)\ 1\ (MULT\ n\ (f\ (PRED\ n)))$$

$$FACT = Y\ G$$

- f: la llamada recursiva (la da Y)
- $ISZERO\ n$: caso base
- $PRED\ n$: $n - 1$

Corrida FACT 3:

$FACT\ 3 = G\ (Y\ G)\ 3$

$\to_\beta IF\ (ISZERO\ 3)\ 1\ (MULT\ 3\ (FACT\ 2))$

$\to MULT\ 3\ (FACT\ 2)$ (3 no es 0)

$\to MULT\ 3\ (MULT\ 2\ (FACT\ 1))$

→ MULT 3 (MULT 2 (MULT 1 (FACT 0)))

→ MULT 3 (MULT 2 (MULT 1 1)) (caso base)

→ 6 ✓

Ciclos: un ciclo while

mientras cond(s): s = paso(s)

se vuelve recursión de cola: el estado s se pasa como parámetro en vez de modificarlo.

$LOOP = Y\ (\lambda l.\lambda s.\ IF\ (cond\ s)\ (l\ (paso\ s))\ s)$

Ejemplo: sumar 1 + 2 + ... + n con acumulador

$SUMA = Y\ (\lambda f.\lambda n.\lambda ac.\ IF\ (ISZERO\ n)\ ac\ (f\ (PRED\ n)\ (ADD\ ac\ n)))$

Corrida SUMA 3 0:

SUMA 3 0 → SUMA 2 3 → SUMA 1 5 → SUMA 0 6 → 6 ✓

cada vuelta del ciclo es una llamada; n y ac son las "variables" del ciclo y nunca se modifican

Ciclo "repetir n veces" con numerales de Church:

$n = \lambda f.\lambda x.\ f\ (f\ (\ldots\ f\ (x)))$ (f aplicada n veces)

$3 = \lambda f.\lambda x.\ f\ (f\ (f\ x))$

$3\ SUCC\ 0 \rightarrow_\beta SUCC\ (SUCC\ (SUCC\ 0)) = 3$

∴ el número ya es un ciclo for de n vueltas

En un lenguaje funcional (Haskell):

```
factorial 0 = 1
factorial n = n * factorial (n - 1)
suma n = foldl (+) 0 [1..n]        -- el "ciclo"
```

map, filter y foldl reemplazan a los for de siempre.

c) ¿Cuándo usar programación funcional y cuándo no?

Es prudente cuando:

- se transforman datos (listas, colecciones) con map, filter, fold
- hay concurrencia o paralelismo: no hay estado mutable compartido ⇒ no hay condiciones de carrera
- se quiere código fácil de probar: misma entrada ⇒ misma salida
- compiladores, parsers y cálculos matemáticos

Ejemplo (sí): total de ventas de noviembre

```
total = sum . map monto . filter ((== 11) . mes)
```

Cada paso es una función pura: se pueden probar por separado y paralelizar (como en MapReduce).
Los problemas 2, 3 y 4 de este lab son de este tipo.

No es prudente cuando:

- el programa es casi todo estado que cambia e I/O (videojuegos)
- hay poca memoria o tiempo real (embebidos): la inmutabilidad crea copias, hay recolector de basura y la recursión gasta pila
- se modifican en el lugar arreglos o matrices enormes
- el equipo no conoce el paradigma

Ejemplo (no): firmware de un microcontrolador (Arduino) que lee un sensor y prende un ventilador:

```
loop: leer T → si T > 30 prender pin → repetir
```

Tiene muy poca RAM, debe responder en tiempo fijo y todo su trabajo es cambiar registros y pines (estado) ⇒ conviene C imperativo.